

# Catch small failures before they become incidents

## THE INDUSTRY PROBLEM AND OUR GOAL

Small errors and timeouts can affect customers without being reported. Left unnoticed, they may grow into major incidents. This POC reads logs to spot pool saturation early, recommends a bounded fix, and verifies the human-approved change with a comparable rerun.

01

### Observe

Simulate 100 users and record outcomes, waits and latency.

02

### Diagnose

Ground two specialist agents in logs and runbooks.

03

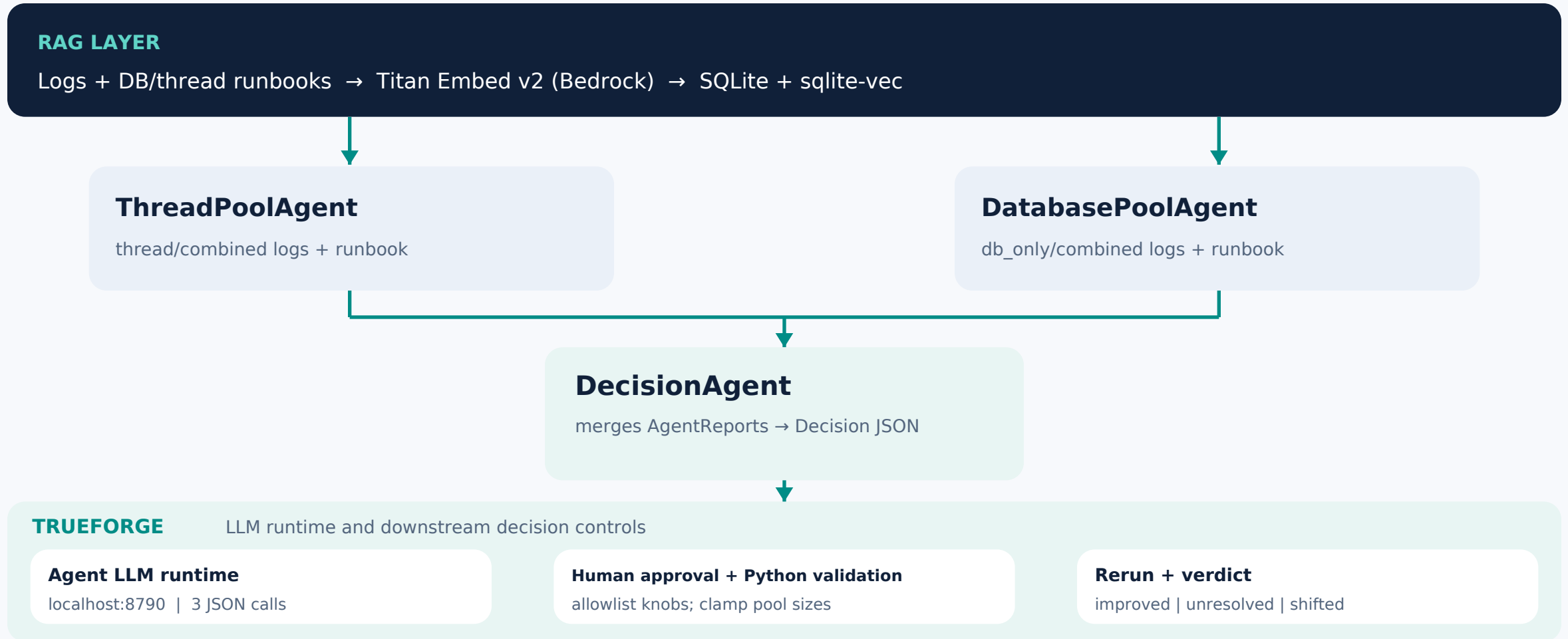
### Verify

Approve bounded changes and rerun the same workload.

#### POC boundary

Local simulator and operator gate; no autonomous production change.

# RAG, specialists, decision and approval



The LLM proposes a change. Python controls what can be applied, and the operator decides whether to proceed.

# How each agent calls the local LLM gateway

The flow per agent call:

```

agent (ThreadPool / DB / Decision)
  |
  v
TrueForge(base_url=cfg.trueforge_base_url) # localhost:8790
  |
  +-- client.sessions.create()             # new session per
      |                                     AgentSpec(
      |   model=Model(name=cfg.llm_model),
      |   instructions=...,                 # system prompt
      |   response_format=ResponseFormatJsonSchema(...)
      | )
      |
      +-- client.sessions.create_turn_stream() # streams the
          |                                     input=[UserMessage(content=prompt)]
          |
          v
streamed events → ModelMessageDeltaEvent → assembled → json.

```

## SDK call path

- 1 Create a new session
- 2 Set model, instructions and JSON schema
- 3 Stream a turn with the agent prompt
- 4 Assemble the streamed JSON result

## Where it fits

ThreadPoolAgent, DatabasePoolAgent and DecisionAgent use this same TrueForge SDK pattern through localhost:8790.

Provided integration screenshot; it illustrates the SDK call structure rather than a production deployment.

# What each phase does

## 1 Workload simulation

100 virtual users; `asyncio.Semaphore` bounds the DB pool and `ThreadPoolExecutor` bounds workers. Each request creates a `LogEntry`; an `asyncio.Queue` feeds one SQLite writer.

## 2 Embedding and retrieval

Titan Embed v2 embeds logs and chunks from two Markdown runbooks. `sqlite-vec` stores searchable vectors; top-k retrieval provides evidence for each agent.

## 3 Specialist and merger agents

`ThreadPoolAgent` and `DatabasePoolAgent` produce structured `AgentReports`. `DecisionAgent` combines both reports and context to propose HYBRID, APPLICATION, DATABASE\_POOL or NO\_ACTION.

## 4 Approval and retest

Python filters to recognized pool settings and clamps values to configured limits. On approval, it reruns the same seed, scenario and users, then compares before and after.

Stop conditions: target success reached, both pools at ceiling, operator rejection, or iteration cap.

# Four runs show a shifting bottleneck

Success: 15% → 60% → 83% → 100%

| Run         | DB pool | Threads | Success | DB TO | Thread TO | p50 ms | p95 ms |
|-------------|---------|---------|---------|-------|-----------|--------|--------|
| Baseline    | 5       | 5       | 15.0%   | 85    | 0         | 1,027  | 2,781  |
| Iteration 1 | 30      | 5       | 60.0%   | 40    | 0         | 4,225  | 5,826  |
| Iteration 2 | 60      | 5       | 83.0%   | 0     | 17        | 6,992  | 7,243  |
| Iteration 3 | 60      | 30      | 100.0%  | 0     | 0         | 4,095  | 7,526  |

**What the results demonstrate**

DB timeouts fell to zero; then thread timeouts appeared. After both pools increased, all 100 requests succeeded.

**Risk still to mitigate**

Final p95 was 7,526 ms vs 2,781 ms at baseline. Repeat runs, set latency and cost limits, and validate under production-like load.